

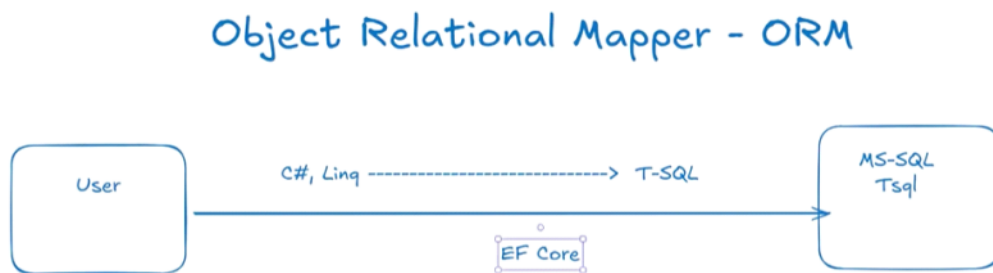
1. What is ORM

31 May 2025 06:52 PM

What is ORM?

ORM (Object-Relational Mapping) is a **programming technique** that allows developers to **interact with a relational database using the object-oriented paradigm** of their programming language (like C#, Java, Python, etc.).

Instead of writing SQL queries to perform database operations, ORM lets you use **objects, classes, and methods** to perform those operations in a more intuitive and readable way.



Popular ORMs

- **Entity Framework, Dapper** (C# / .NET)
- **Hibernate** (Java)
- **Mongoose** (Node.js app to MongoDB)
- **SQLAlchemy** (Python)
- **Django ORM** (Python)
- **ActiveRecord** (Ruby on Rails)

If:

- You want simplicity and raw speed: **Dapper**
- You want full ORM features like migrations, LINQ, change tracking: **Entity Framework**

2. Managing connection strings

Saturday, March 1, 2025

1:48 PM

1. Using appsettings.json

Example appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=MyDatabase;User Id=myuser;Password=mypassword;"
  }
}
```

Accessing the Connection String in Startup.cs

```
public class Startup
{
    private readonly IConfiguration _configuration;

    public Startup(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public void ConfigureServices(IServiceCollection services)
    {
        string connectionString = _configuration.GetConnectionString("DefaultConnection");

        services.AddDbContext<ApplicationDbContext>(options =>
            options.UseSqlServer(connectionString));
    }
}
```

2. Azure Key Vault (for Production and Cloud Environments)

For production environments, especially in the cloud, it's a best practice to store sensitive data, such as connection strings, in a **secure vault** like **Azure Key Vault**. Azure Key Vault allows you to securely store and access secrets, certificates, and connection strings.

Steps to use Azure Key Vault in .NET:

1. **Set up Azure Key Vault:** Create a Key Vault in Azure, and add a secret (e.g., your database connection string).
2. **Add Azure Key Vault NuGet package:**

```
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add package Azure.Identity
```

Configure Key Vault in Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add Azure Key Vault to the configuration
builder.Configuration.AddAzureKeyVault(new Uri("https://<your-keyvault-name>.vault.azure.net/"), new
DefaultAzureCredential());

// Access the connection string from Key Vault
var connectionString = builder.Configuration["ConnectionStrings:DefaultConnection"];

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

var app = builder.Build();
```

3. Database Connection String for Docker

If you're using Docker, you can set the connection string as an **environment variable** in the Docker container configuration

Example docker-compose.yml

```
version: '3.4'

services:
  myapp:
    image: myapp:latest
    environment:
      - ConnectionStrings__DefaultConnection=Server=mydb;Database=MyDatabase;User
        Id=myuser;Password=mypassword;
```

3. EF Core Database First Approach

Monday, February 17, 2025 6:52 PM

1. Create a Project (if you haven't already)

If you're starting from scratch, create a .NET project (Console App, Web API, MVC, etc.).

2. Install EF Core NuGet Packages

```
dotnet add package Microsoft.EntityFrameworkCore
```

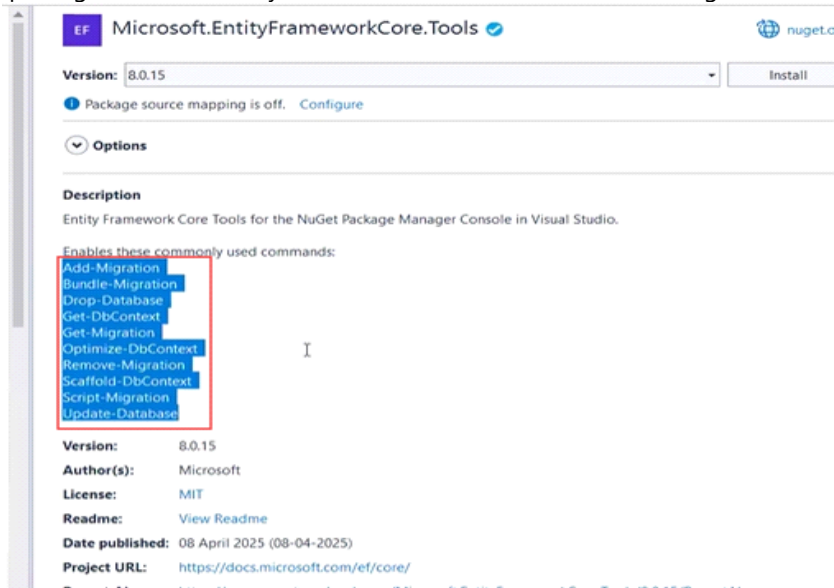
```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer # If using SQL Server
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools # For tools support (e.g., migrations)
```

Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

package Microsoft.EntityFrameworkCore.Tools enables following commands



Scaffold the Database

To generate the models and DbContext class based on the existing database schema, you use the **EF Core Scaffold-DbContext** command.

The general syntax for this command is:

```
Scaffold-DbContext "YourConnectionString" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context YourDbContextName
```

```
-Connection "Data Source = . ; Initial Catalog = QuickKartDB ; Integrated Security = true"
```

Server name Database name Windows credentials

Dot (.) for local machine

Inspect the Generated Classes

After running the scaffold command, EF Core generates:

- **DbContext** class: This class will represent the session between your application and the database. It includes `DbSet<TEntity>` properties for each table in your database, and it is responsible for querying and saving instances of your entities.
- **Entity Classes**: These classes represent the tables in your database. Each entity class will have properties corresponding to the columns in the respective table. EF Core uses conventions to map these properties to the database columns. For example, if a table has a column `FirstName`, the class will have a property `FirstName`.

Managing Changes to the Database

Since you're using the **Database First** approach, if there are changes to the database schema (e.g., new tables, changes in existing tables), you will need to regenerate the models to reflect those changes.

You can re-run the `Scaffold-DbContext` command after making schema changes:


```
dotnet ef dbcontext scaffold "YourConnectionString" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --context YourDbContext --context-dir Data --force
```

This will overwrite the existing models and DbContext with updated versions that reflect the new database schema.

You can also just scaffold specific tables using the -Tables flag to avoid regenerating everything:

```
dotnet ef dbcontext scaffold "YourConnectionString" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --context YourDbContext --context-dir Data --force --tables Product
```

Migrations (Optional): If you need to add any new schema changes or update the database schema, you can manually apply migrations. However, with the Database First approach, the main focus is on querying and working with an already existing schema. If you want to apply any new changes, you'll need to use migration commands.

```
public class QuickKartRepository
{
    private QuickKartDbContext context;
    public QuickKartRepository(QuickKartDbContext context)
    {
        this.context = context;
    }

    public List<Product> GetProductsOnCategoryId(byte categoryId)
    {
        List<Product> lstProducts = new List<Product>();
        try
        {
            lstProducts = context.Products.Where(p => p.CategoryId == categoryId).ToList();
        }
        catch (Exception ex)
        {
            lstProducts = null;
        }
        return lstProducts;
    }

    public bool AddCategory(string categoryName)
    {
        bool status = false;
        Category category = new Category();
        category.CategoryName = categoryName;
        try
        {
            context.Categories.Add(category);
            //context.Add<Category>(category);
            context.SaveChanges();
            status = true;
        }
        catch (Exception ex)
        {
            status = false;
        }
        return status;
    }

    public bool DeleteProduct(string productId)
    {
        Product product = new Product();
        bool status = false;
        try
        {
            product = context.Products.Find(productId);
        }
    }
}
```

```

        if (product != null)
        {
            context.Products.Remove(product);
            context.SaveChanges();
            status = true;
        }
        else
        {
            status = false;
        }
    }
    catch (Exception)
    {
        status = false;
    }
    return status;
}
}

```

How to change approach from DB first => Code First

1. After scaffold Database
2. Run add-migration command, it will create initial migration
3. Remove fluent validation from Up and Down method. For Ex it will look.-


```

protected override void Up(MigrationBuilder migrationBuilder)
{

}

protected override void Down(MigrationBuilder migrationBuilder)
{

}

```
4. Run update-database command
5. New migration generate in migrationHistory.

4. EF Core Code First Approach

07 June 2025 10:59 PM

Required NuGet Packages

Install these EF Core packages:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Step

1. Create Model Classes

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }

    // Foreign key
    public int CategoryId { get; set; }
    public Category Category { get; set; }
}

public class Category
{
    public int Id { get; set; }
    public string CategoryName { get; set; }
    public List<Product> Products { get; set; }
}
```

2. Create DbContext class

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Fluent API (optional customization)
        modelBuilder.Entity<Product>()
            .Property(p => p.Name)
            .IsRequired()
            .HasMaxLength(100);
    }
}
```

3. Configure in Program.cs

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
```

```
?? throw new InvalidOperationException("Connection string"
+ "'DefaultConnection' not found.");
));
```

4. Add Migration

```
add-migration <MigrationName>
```

5. Update Database

```
Update-database
```

```
MyApp/
├── Migrations/
│   ├── 20240607183000_InitialCreate.cs
│   └── AppDbContextModelSnapshot.cs
├── Models/
│   ├── Product.cs
│   └── Category.cs
├── Data/
│   └── AppDbContext.cs
├── Program.cs
└── appsettings.json
```

Revert Last Applied Changes

Run this to **list available migrations**:

```
-- Get-Migrations
```

Pick the **migration before the last one**

```
-- Update-Database <nameOfPreviousMigration>
```

Remove current migration

```
-- Remove-Migration
```

5. DbContext class

31 May 2025 08:32 PM

1. Create Model Classes (Entities)

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

2. Create the DbContext Class

```
using Microsoft.EntityFrameworkCore;

public class SchoolContext : DbContext
{
    // Define DbSet properties for each model (table)
    public DbSet<Student> Students { get; set; }

    // Configure the database connection
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer("Server=.;Database=SchoolDB;Trusted_Connection=True;");
    }
}
```

3. Alternative: Use Dependency Injection (in ASP.NET Core)

Instead of OnConfiguring(), use constructor injection in ASP.NET Core:

```
public class SchoolContext : DbContext
{
    public SchoolContext(DbContextOptions<SchoolContext> options) : base(options) { }

    public DbSet<Student> Students { get; set; }
}
```

Then in Program.cs or Startup.cs:

```
builder.Services.AddDbContext<SchoolContext>(options =>
    options.UseSqlServer("Your_Connection_String"));
```

Part	Description
DbSet<Student>	Represents the Students table
OnConfiguring()	Sets up database connection if not using DI
UseSqlServer()	Chooses SQL Server provider (others include UseSqlite(), UseNpgsql() for PostgreSQL)

6. Model Configuration Techniques

07 June 2025 11:29 PM

- 1. Data Annotations
- 2. Fluent API

Separate Fluent API Configuration File

Separate Configuration Class (Recommended for large projects)

This keeps your code clean and modular.

Step 1: Create a config class

```
public class ProductConfiguration : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> builder)
    {
        builder.ToTable("tbl_Products");
        builder.Property(p => p.Name)
            .IsRequired()
            .HasMaxLength(100);
    }
}
```

Step 2: Register in OnModelCreating

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfiguration(new ProductConfiguration());
}
```

3. Automatically Apply All Config Classes

If you have many configuration files, use this:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(YourDbContext).Assembly);
}
```

◊ This scans the entire assembly and applies all IEntityTypeConfiguration<T> implementations automatically.

Method	Description / Purpose	Example
HasKey()	Defines the primary key (supports composite keys).	builder.HasKey(e => e.Id); builder.HasKey(e => new { e.OrderId, e.ProductId });
ToTable("TableName")	Maps the entity to a custom table name in the database.	builder.ToTable("tbl_Products");
Property(x => x.PropName)	Accesses a property to configure its mapping.	builder.Property(p => p.Name);
HasColumnName("ColumnName")	Sets the column name in the database for a given property.	builder.Property(p => p.Name).HasColumnName("ProductName");
HasColumnType("sqlType")	Specifies the SQL data type (e.g., varchar(50), decimal(18,2)).	builder.Property(p => p.Price).HasColumnType("decimal(18,2)");
IsRequired()	Marks the property as NOT NULL in the database.	builder.Property(p => p.Name).IsRequired();
HasMaxLength(n)	Sets the maximum allowed length for a string column.	builder.Property(p => p.Name).HasMaxLength(50);
HasDefaultValue(value)	Specifies a default value for a column in the database.	builder.Property(p => p.IsActive).HasDefaultValue(true);
HasDefaultValueSql("SQL")	Specifies a default value using a SQL expression (e.g., GETDATE()).	builder.Property(p => p.CreatedAt).HasDefaultValueSql("GETDATE()");
HasComputedColumnSql("SQL")	Marks the column as computed using a SQL formula (e.g., full name from first + last).	builder.Property(e => e.FullName).HasComputedColumnSql("[FirstName] + ' ' + [LastName]");
HasIndex(x => x.Prop)	Defines an index on one or more columns.	builder.HasIndex(p => p.Name).HasDatabaseName("IX_Product_Name");
IsUnique()	Specifies that the index should be unique.	builder.HasIndex(p => p.Email).IsUnique();
HasOne() / WithOne()	Configures a one-to-one relationship between entities.	builder.HasOne(u => u.Profile).WithOne(p => p.User).HasForeignKey<Profile>(p => p.UserId);
HasOne() / WithMany()	Configures a one-to-many relationship (reference to	builder.HasOne(p => p.Category).WithMany(c =>

	collection).	c.Products).HasForeignKey(p => p.CategoryId);
HasMany() / WithOne()	Configures a many-to-one relationship (collection to reference).	builder.HasMany(c => c.Products).WithOne(p => p.Category);
HasForeignKey(x => x.FK)	Specifies the foreign key property explicitly.	builder.HasOne(p => p.Category).WithMany(c => c.Products).HasForeignKey(p => p.CategoryId);
UsingEntity<JoinType>()	Configures the join entity in many-to-many relationships.	builder.HasMany(u => u.Roles).WithMany(r => r.Users).UsingEntity<UserRole>();
OwnsOne(x => x.Prop)	Configures an owned (value) type within an entity.	builder.OwnsOne(p => p.Address);
Property<string>("ShadowProp")	Declares a shadow property (not in class, tracked by EF).	builder.Property<DateTime>("CreatedOn");
HasConversion<T>()	Converts a property to another type for storage (e.g., enum to string).	builder.Property(u => u.Status).HasConversion<string>();
HasQueryFilter(x => x.IsActive)	Applies a global filter for all queries (e.g., for soft delete).	builder.HasQueryFilter(p => !p.IsDeleted);
Ignore(x => x.Prop)	Excludes a property from being mapped to the database.	builder.Ignore(p => p.TempData);
ApplyConfiguration(new Config())	Applies external configuration via IEntityTypeConfiguration<T>.	modelBuilder.ApplyConfiguration(new ProductConfig());

Attribute	Property/Usage	Description	Example
[Key]	Primary key	Specifies the primary key of the entity.	[Key] public int Id { get; set; }
[Required]	Validation	Specifies that a property is required (non-nullable).	[Required] public string Name { get; set; }
[MaxLength]	Max string/array length	Sets the maximum length allowed.	[MaxLength(50)] public string Description { get; set; }
[MinLength]	Min string/array length	Sets the minimum length allowed.	[MinLength(5)] public string Username { get; set; }
[StringLength]	Exact string length	Combines MaxLength and MinLength in one attribute.	[StringLength(50, MinimumLength = 5)]
[Column]	Column name and type	Maps a property to a specific column name and SQL data type.	[Column("ProductName", TypeName = "varchar(50)")]
[Table]	Table name	Maps an entity to a specific table in the database.	[Table("tbl_Products")]
[ForeignKey]	Foreign key relationship	Indicates a property is a foreign key.	[ForeignKey("CategoryId")] public Category Category { get; set; }
[InverseProperty]	Relationship direction	Defines the inverse navigation property in a relationship.	[InverseProperty("Manager")] public ICollection<Employee> TeamMembers { get; set; }
[NotMapped]	Exclude property	Indicates a property is not mapped to any database column.	[NotMapped] public string TempData { get; set; }
[ConcurrencyCheck]	Optimistic concurrency	Marks property for concurrency check.	[ConcurrencyCheck] public int Version { get; set; }
[Timestamp]	Timestamp concurrency	Used for row versioning with a byte[] timestamp.	[Timestamp] public byte[] RowVersion { get; set; }
[DatabaseGenerated]	Identity/Computed	Controls how values are generated (None, Identity, Computed).	[DatabaseGenerated(DatabaseGeneratedOption.Identity)] public int Id { get; set; }
[Index]	Index creation (EF Core 7+)	Creates an index on a column.	[Index(nameof(Name), IsUnique = true)]
[Precision]	Decimal precision	Defines the precision and scale of a decimal column.	[Precision(10, 2)] public decimal Price { get; set; }
[DefaultValue]	Set default value	Sets a default value for a column.	[DefaultValue("GETDATE()")] public DateTime CreatedAt { get; set; }

7. Entity Relationship

09 June 2025

10:46 PM

- **One-to-One**
- **One-to-Many**
- **Many-to-Many**

1 One-to-One Relationship

✎ *One record in Table A corresponds to one record in Table B.*

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public Address Address { get; set; } // Navigation property
}

public class Address
{
    [Key]
    public int StudentId { get; set; } // Primary Key and also Foreign Key
    public string City { get; set; }

    public Student Student { get; set; }
}
```

Fluent API Configuration:

```
modelBuilder.Entity<Student>()
    .HasOne(s => s.Address)
    .WithOne(a => a.Student)
    .HasForeignKey<Address>(a => a.StudentId);
```

2 One-to-Many Relationship

✎ *One record in Table A can have multiple related records in Table B.*

```
public class Department
{
    public int DepartmentId { get; set; }
    public string DeptName { get; set; }

    public ICollection<Employee> Employees { get; set; }
}

public class Employee
{
    public int EmployeeId { get; set; }
    public string EmpName { get; set; }

    public int DepartmentId { get; set; }
    public Department Department { get; set; }
}
```


Fluent API Configuration:

```
modelBuilder.Entity<Employee>()  
    .HasOne(e => e.Department)  
    .WithMany(d => d.Employees)  
    .HasForeignKey(e => e.DepartmentId);
```

3 Many-to-Many Relationship

 ***Multiple records in Table A can relate to multiple records in Table B.***

```
public class Course  
{  
    public int CourseId { get; set; }  
    public string Title { get; set; }  
  
    public ICollection<Student> Students { get; set; }  
}  
  
public class Student  
{  
    public int StudentId { get; set; }  
    public string Name { get; set; }  
  
    public ICollection<Course> Courses { get; set; }  
}
```

Fluent API Configuration:

```
modelBuilder.Entity<Course>()  
    .HasMany(c => c.Students)  
    .WithMany(s => s.Courses)  
    .UsingEntity(j => j.ToTable("StudentCourses"));
```

8. Executing Stored Procedures and SQL Functions in EF Core

08 June 2025 10:28 PM

In **Entity Framework Core**, you can call **Stored Procedures**, **Functions**, and execute **Raw SQL** using:

Calling a Stored Procedure that returns rows

Suppose you have this stored procedure:

```
-- SQL SP
CREATE PROCEDURE GetActiveProducts
AS
BEGIN
    SELECT * FROM Products WHERE IsActive = 1
END
```

```
var products = context.Products
    .FromSqlRaw("EXEC GetActiveProducts")
    .ToList();
```

If parameters:

```
var categoryId = 2;
var products = context.Products
    .FromSqlRaw("EXEC GetProductsByCategory @CategoryId = {0}", categoryId)
    .ToList();
```

Or with interpolation:

```
var products = context.Products
    .FromSqlInterpolated($"EXEC GetProductsByCategory {categoryId}")
    .ToList();
```

For INSERT/UPDATE/DELETE/Non-query SPs

Used when the stored procedure **does not return a result set**, only affects rows.

```
-- SQL SP
CREATE PROCEDURE UpdateProductPrice
    @ProductId INT,
    @NewPrice DECIMAL(10,2)
AS
BEGIN
    UPDATE Products SET Price = @NewPrice WHERE Id = @ProductId
END
```

```
var productId = 1;
var newPrice = 199.99m;

context.Database.ExecuteSqlRaw("EXEC UpdateProductPrice @p0, @p1", productId, newPrice);
```

Or with interpolation:

```
context.Database.ExecuteSqlInterpolated($"EXEC UpdateProductPrice {productId}, {newPrice}");
```

```
var products = context.Products
```

```
.FromSqlRaw("SELECT * FROM Products WHERE Price > {0}", 1000)  
.ToList();
```

```
int affectedRows = context.Database.ExecuteSqlRaw("UPDATE Products SET Price = Price * 1.1 WHERE CategoryId = {0}", 3);
```

Summary Table

Method	Use For	Returns	Usage
FromSqlRaw()	SELECT / SP with result set	IQueryable<T>	Read operations
ExecuteSqlRaw()	INSERT / UPDATE / DELETE	int(rows affected)	Non-query SPs or commands
HasDbFunction()	UDFs (scalar functions)	LINQ usable result	Function mapping

9. Exceptions and Errors

23 August 2025 08:45 PM



EF Core: A second operation was started on this context Error

- **Cause:**

DbContext is **not thread-safe**. Error happens when:

- Running multiple queries in parallel on the same DbContext.
- Or using DbContext as **Singleton** instead of **Scoped**.
- I requested again before complete current request.

Exception -

System.ObjectDisposedException: 'Cannot access a disposed context instance. A common cause of this error is disposing a context instance that was resolved from dependency injection and then later trying to use the same context instance elsewhere in your application. This may occur if you are calling 'Dispose' on the context instance, or wrapping it in a using statement. If you are using dependency injection, you should let the dependency injection container take care of disposing context instances

Soluton - I was using AddAsync(), SaveChangesAsync(), CommitAsync() without await. So when this 3 method still working with DB, the return was executed immediately and it disposed the db context.

```
public async Task<string> MakePaymentAsync(PaymentDetailRequestDto requestDto)
{
    var paymentDetail = _mapper.Map<PaymentDetail>(requestDto);
    if (paymentDetail is not null)
    {
        var Order = _mapper.Map<Order>(requestDto.OrderRequest);
        await _orderRepository.AddAsync(Order);
    }

    await _paymentDetailRepository.AddAsync(paymentDetail);
    await _paymentDetailRepository.CommitAsync();
    return await Task.FromResult("Payment Successful");
}
```

So what happened?

- ☞ The method **returned immediately**
- ☞ The HTTP request **finished**
- ☞ DI **disposed DbContext**
- ☞ Async DB operations were **still running**
- ☞ ✨ **ObjectDisposedException**

Why async methods MUST be awaited

Async methods like:

```
AddAsync()  
SaveChangesAsync()  
CommitAsync()
```

☞ Run **asynchronously**

☞ They return a **Task**

☞ They continue execution **after** the current method exits if not awaited

So without await:

```
// Fire and forget (VERY dangerous with DbContext)  
_orderRepository.AddAsync(Order);
```

✓ What changed internally

1. Method **waits** for DB operations
2. Request **does NOT finish early**
3. DbContext stays **alive**
4. SaveChanges executes safely
5. No disposed context error 🐛

Never call async EF Core methods without await

One-line explanation (for interviews / notes)

Without await, async database operations continue after the request ends, causing DbContext to be disposed while still in use.

```
DoAsync();    // ✗ starts and forgets  
await DoAsync(); // ☑ waits and continues safely
```